

LLM Queries Summary:

I started by looking through the Spider benchmark: taking several different databases and asking GPT-4 to generate responses to their associated benchmark natural language queries. I generated a benchmark dataset of 50 queries (approximately half true and half false). I then tried different prompts and models.

Baseline prompt:

```
messages=[  
    {"role": "system", "content": "You are an expert SQL translator and verifier."},  
    {"role": "system", "content": "First, determine whether the SQL query is correct or incorrect and provide no explanation. Second, if the query is incorrect, also provide an explanation as to why."},  
    {"role": "user", "content": f"Here is the schema of the database: {schema_str}"},  
    {"role": "user", "content": f"Here is a natural language query {natural_language_query}. It has been translated to SQL as follows: {llm_sql}. TRUE or FALSE: does the SQL response correctly answer the natural language query?"},  
]
```

Different strategies I tried:

- Explicitly stating that the queries were syntactically correct but may or may not be logically correct (*as many of the initial errors had to do with the LLM perceiving a syntax error where there wasn't any*)
- Asking the LLM to first tell me what the SQL query was doing, then whether it correctly answered the natural language query
 - o *This didn't have much success.*
- Providing the LLM with the results of the original natural language query
 - o *This was especially helpful in determining queries that returned duplicate results. If the number of duplicate rows was 3+ and the duplicates were immediately after one another, the LLM could detect duplicates. If there were only two duplicates or they were spread apart in the results, the LLM could not always detect issues with the query.*
- If the LLM marked the query as false, asking it to generate a new query, then running that new query on the dataset and asking it to compare the original results to the new ones.
 - o *This worked well for false negative queries, where the original query was correct but the LLM marked it as false. Often the LLM-generated query would return the same results as the original query, and the LLM would see that its modifications did not make a difference. However, if the LLM-generated query was incorrect, returning the results—which would also be false—could confuse the LLM further.*

- Asking the LLM to list all the issues it saw with the query, and then create a new query that addressed all of these issues.
 - o *Tested this in Playground: this seemed to work well for generating new correct queries.*
- Trying different models: CodeLLama, GPT-4o, GPT-4
 - o *GPT-4o had the best performance against the benchmark.*

Final prompt with best performance:

messages=[

 {"role": "system", "content": "You are an expert SQL translator and verifier."},

 {"role": "system", "content": "First, determine whether the SQL query is correct or incorrect. Second, provide an explanation as to why."},

 {"role": "system", "content": "The following is very important: my career depends on it. Assume all SQL queries are syntactically correct but may or may not be logically correct."},

 {"role": "user", "content": f"{schema_str}"},

 {"role": "user", "content": f"Here is a natural language query {natural_language_query}. It has been translated to SQL as follows: {llm_sql}. Here is the first 500 characters of the SQL query when executed against the database: {truncated_result_str}. "},

 {"role": "user", "content": "TRUE or FALSE (and give an explanation): does the SQL response correctly answer the natural language query?"}

]

I then added some queries from the BIRD benchmark, which didn't have a significant impact on the overall benchmark accuracy.

I also created a separate benchmark with queries from a synthetic "episodes" dataset. These queries were much more complex than in Spider, with multiple CTEs and many more tables. The above prompt's performance on this benchmark varied from 50% to 70%. A large number of the misclassifications were due to false negatives.

The LLM-perceived errors are listed below:

1. Join logic:

- o The `CASE` statement in the `GROUP BY` clause is correct, but the `COUNT(\*)` aggregates all objects without considering the specific episodes.
- o The join condition `episode.video\_name = evo.video\_name` is incorrect. The `episode` table does not have a direct `video\_name` column that can be used to join with `episodevideos\_objects`. Instead, the join should be based on the `episode\_id` and `video\_id` relationship.

2. Potential issue with non-unique video_names. Makes sense for the synthetic dataset but could be an issue for the actual dataset.

- It only checks if the `video\_name` is in the `Top2Episodes`, which might not be sufficient if `video\_name` is not unique across episodes.
- The main issue lies in the join logic. The query joins `episodevideos` with `episode` on `video\_name`, which is not a direct relationship. The correct join should be based on `episode\_id` to ensure that the objects are correctly associated with their respective episodes.
- If there are inconsistencies or mismatches in the `video\_name` references between the `episode`, `episodevideos`, and `episodevideos\_objects` tables, the joins might fail, leading to incorrect or incomplete data aggregation.

3. Episodes without associated objects, without engagement data, with multiple videos/duplicate entries:

- If there are episodes in the `episode` table that do not have corresponding entries in the `episodeengagement` table, those episodes will be excluded from the `RankedEpisodes` CTE. This could lead to an incomplete ranking and potentially incorrect identification of the top 2 episodes.
- If the top 2 episodes do not have any associated objects in the `episodevideos\_objects` table, the `ObjectCounts` CTE will not have a row for 'top_2'. This will cause the final `SELECT` statement to return `NULL` for `top\_2\_count`.
- If an episode is associated with multiple videos, the current join logic might not correctly aggregate object counts across all videos for that episode. This could lead to undercounting or overcounting objects for such episodes.

4. Ties:

- If there are more than three videos with the same maximum duration, the `LIMIT 3` clause will arbitrarily select three of them. This could lead to inconsistent results if the dataset changes slightly.

5. Inconsistent data:

- If the `video\_name` in the `episodevideos` table does not match exactly with the `video\_name` in the `episodeengagement` table (e.g., due to leading/trailing spaces or case sensitivity), the `JOIN` operation will fail to match those rows.
- If the `duration` column is not stored as an integer or a comparable numeric type, the `ORDER BY` clause might not sort the durations correctly.

Issue #2 was solvable in Playground by explicitly prompting with “Assume that in this database the video_name is a unique identifier.”

#3 was due to incorrect logic in the LLM: excluding these entries would fit the queries provided and would be expected behavior. The LLM only brought up these issues when explicitly prompted for potential errors, however, not when running the original query.

#4 can be addressed by modifying the original query to define behavior for ties (picking the first three entries lexicographically, etc.).

For these more complex queries, the LLM can occasionally get confused: for example, if episodes are ordered by engagement level in the first CTE, the LLM can get confused about whether the

episodes are still ordered in the second CTE and mark the query as incorrect. Asking the LLM to walk through each CTE can be helpful in this case.

Most of these false positives can be addressed in Playground by asking the LLM to generate a new SQL query and demonstrating that the results of that query match the results of the original query. This can occasionally require some back and forth between the human and the LLM, so I didn't figure out a way to standardize the process, but it is doable.